
IIC2523

Sistemas Distribuidos

— Hernán F. Valdivieso López —
(2026 - 2 / Clase 11)

Replicación de Datos

¿Cómo aseguramos consistencia en las réplicas?

Temas de la clase

1. Introducción
 - a. ¿Por qué replicar los datos?
 - b. Modelos de Consistencia
2. Modelos de consistencia Secuencial
 - a. ¿En qué consiste?
 - b. Protocolo de replicación Activa (*Replicated-Write Protocols*)
3. Modelos de consistencia Fuerte (Linealización)
 - a. ¿En qué consiste?
 - b. Protocolo de replicación Pasiva (*Primary-Based Protocols*)

Introducción

¿Por qué replicar los datos?

Modelos de Consistencia

¿Por qué replicar la información?

- ◆ En un sistema distribuido, la replicación de información permite ofrecer una serie de propiedades.
- ◆ **Alta Disponibilidad (*High Availability*):**
 - ◆ La replicación garantiza que si un nodo falla, el servicio general puede permanecer disponible para los usuarios al tener copias de los datos en otros lugares.
 - ◆ Esto también ayuda a ser tolerante a fallos.

¿Por qué replicar la información?

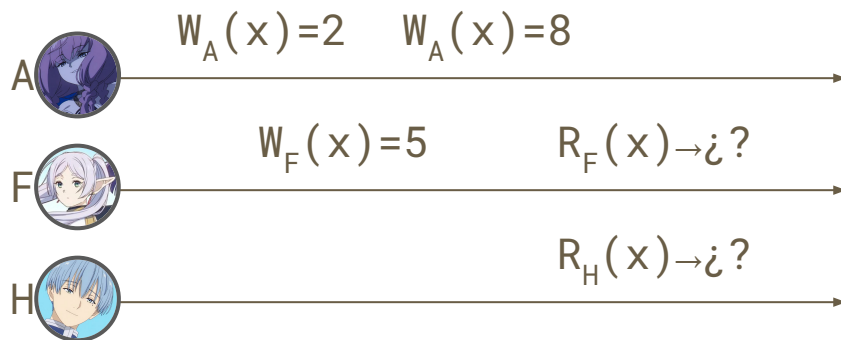
- ◆ En un sistema distribuido, la replicación de información permite ofrecer una serie de propiedades.
- ◆ **Rendimiento y Escalabilidad (*Performance and Scalability*):**
 - ◆ **Reducción de Latencia:** Al tener copias de los datos más cerca de los clientes (físicamente o en la red), se reduce el tiempo de respuesta para las operaciones de lectura.
 - ◆ **Balanceo de Carga:** La replicación permite distribuir las solicitudes de lectura y escritura entre múltiples servidores, evitando cuellos de botella en un único punto.

¿Por qué replicar la información?

- ◆ En un sistema distribuido, la replicación de información permite ofrecer una serie de propiedades.
- ◆ **Operación Desconectada (*Disconnected Operation*):**
 - ◆ Permite a los clientes (por ejemplo, en dispositivos móviles) seguir trabajando con sus copias locales de datos incluso cuando no hay conectividad de red, y luego sincronizar los cambios al reconectarse.

¿Por qué replicar la información? Desafíos

- ◆ La replicación introduce un problema... cada vez que se actualiza una réplica, esta se diferencia de las demás.
 - ◆ Sin reloj global preciso es difícil identificar la "última" operación de escritura.



¿Por qué replicar la información? Desafíos

- ◆ La replicación introduce un problema... cada vez que se actualiza una réplica, esta se diferencia de las demás.
 - ◆ Sin reloj global preciso es difícil identificar la "última" operación de escritura.
- ◆ Es necesario propagar las actualizaciones con algún tipo de reglas de tal manera que las inconsistencias temporales no se noten.
 - ◆ Pero... esta propagación puede degradar severamente el rendimiento, especialmente en sistemas distribuidos a gran escala.

¿Por qué replicar la información? Desafíos

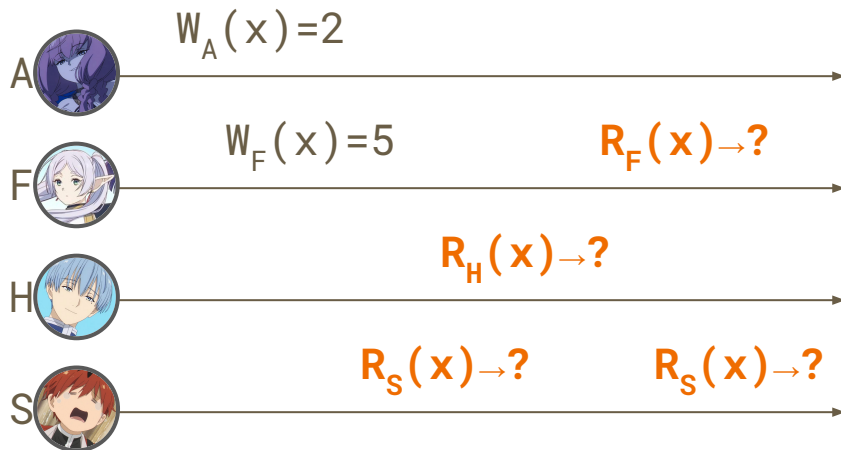
- ◆ La replicación introduce un problema... cada vez que se actualiza una réplica, esta se diferencia de las demás.
 - ◆ Sin reloj global preciso es difícil identificar la "última" operación de escritura.
- ◆ Es necesario propagar las actualizaciones con algún tipo de reglas de tal manera que las inconsistencias temporales no se noten.
 - ◆ Pero... esta propagación puede degradar severamente el rendimiento, especialmente en sistemas distribuidos a gran escala.
- ◆ La solución a este problema es relajar la consistencia en cierta medida.
 - ◆ Esto produce la necesidad de diferentes **modelos de consistencia**.

Modelos de Consistencia

- ◆ Un modelo de consistencia es esencialmente un contrato entre los nodos y la base de datos.
 - ◆ Si los nodos aceptan obedecer ciertas reglas, la base de datos promete funcionar consistentemente.
- ◆ Los modelos se basan principalmente en cómo deben funcionar las operaciones que modifican algún dato (*write*) y operaciones de lecturas (*read*).

Modelos de Consistencia

- ◆ Un modelo de consistencia es esencialmente un contrato entre los nodos y la base de datos.
 - ◆ Si los nodos aceptan obedecer ciertas reglas, la base de datos promete funcionar consistentemente.
- ◆ Los modelos se basan principalmente en cómo deben funcionar las operaciones que modifican algún dato (*write*) y operaciones de lecturas (*read*).



Los modelos de consistencia permiten determinar qué valores retornan los *Read* para garantizar alguna propiedad.

Modelos de Consistencia

Reglas base para analizar un modelo de consistencia

1. Cuando un nodo hace *write* de un valor, se entiende que este nodo **ya conoce** el valor definido. Análogo a que hizo un *read* de forma implícita de dicho valor.
2. La propagación de algún *write* de un nodo a otro solo se confirma cuando el otro nodo hace *read* del dato.
 - a. Si el nodo B hace un *write*, este fue propagado y procesado por el nodo A sólo cuando este último haga *read* de la variable analizada y lea el valor correspondiente.

Modelos de Consistencia

Reglas base para analizar un modelo de consistencia

1. Cuando un nodo hace *write* de un valor, se entiende que este nodo **ya conoce** el valor definido. Análogo a que hizo un *read* de forma implícita de dicho valor.
2. La propagación de algún *write* de un nodo a otro solo se confirma cuando el otro nodo hace *read* del dato.
 - a. Si el nodo B hace un *write*, este fue propagado y procesado por el nodo A sólo cuando este último haga *read* de la variable analizada y lea el valor correspondiente.
3. **Se asumirá que todos los modelos de consistencia se cumplen a no ser que encontremos evidencia explícita que rompa alguna regla del modelo.**

Modelo de Consistencia Secuencial

¿En qué consiste?

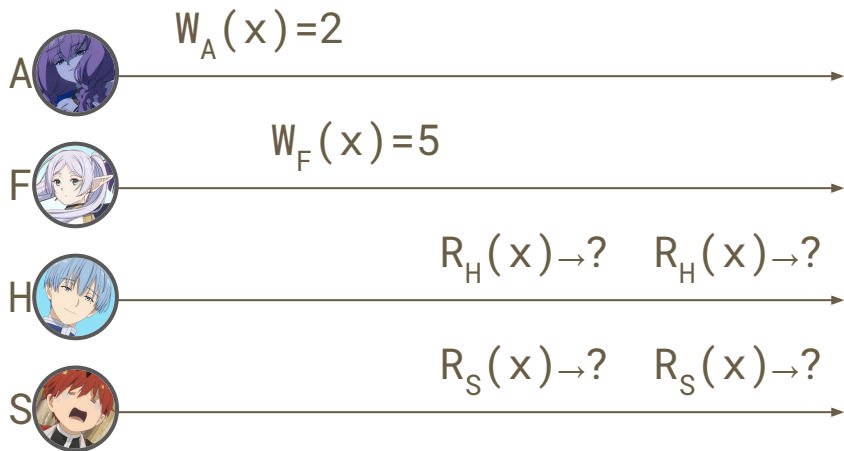
Protocolo de replicación Activa
(*Replicated-Write Protocols*)

Consistencia Secuencial [Lamport, 1979]

- ◆ El resultado de cualquier ejecución es el mismo que si las operaciones de todos los nodos se ejecutarán **en algún orden secuencial** ... y cada nodo individual respeta dicho orden.
 - ◆ Cualquier intercalación válida de las operaciones entre nodos es aceptable... pero todos son **consistentes con esa** misma intercalación.

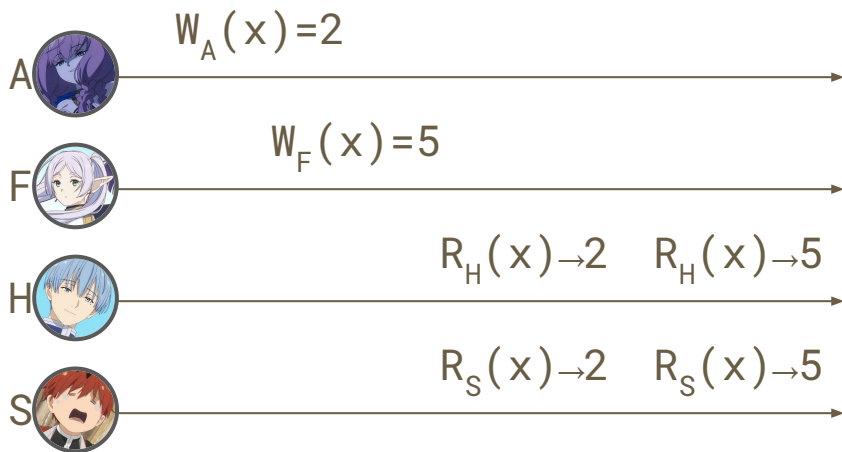
Consistencia Secuencial [Lamport, 1979]

- ◆ El resultado de cualquier ejecución es el mismo que si las operaciones de todos los nodos se ejecutarán **en algún orden secuencial** ... y cada nodo individual respeta dicho orden.
 - ◆ Cualquier intercalación válida de las operaciones entre nodos es aceptable... pero todos son **consistentes con esa** misma intercalación.



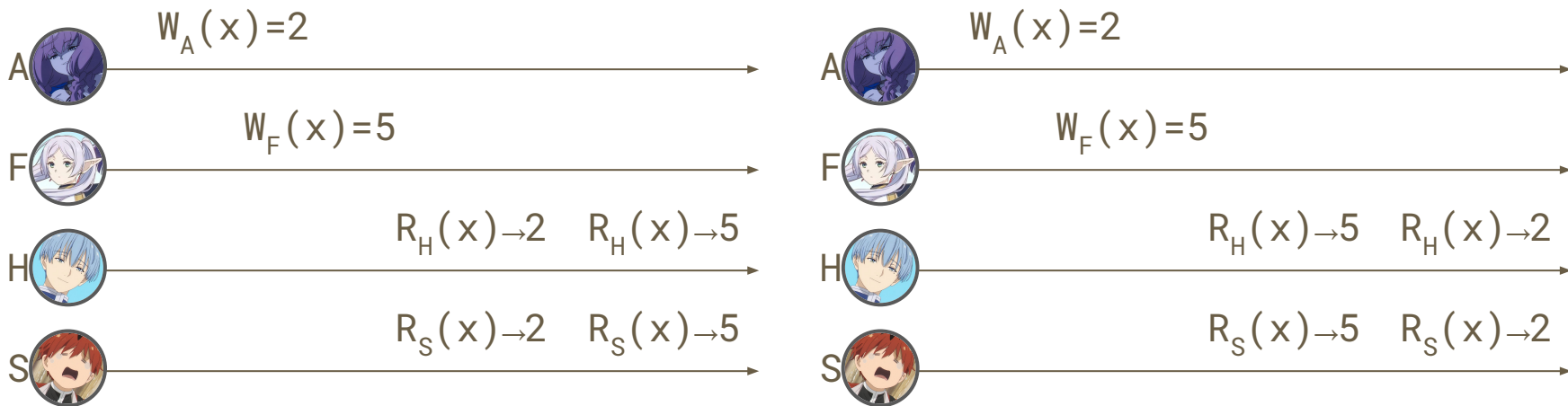
Consistencia Secuencial [Lamport, 1979]

- ◆ El resultado de cualquier ejecución es el mismo que si las operaciones de todos los nodos se ejecutarán **en algún orden secuencial** ... y cada nodo individual respeta dicho orden.
 - ◆ Cualquier intercalación válida de las operaciones entre nodos es aceptable... pero todos son **consistentes con esa** misma intercalación.



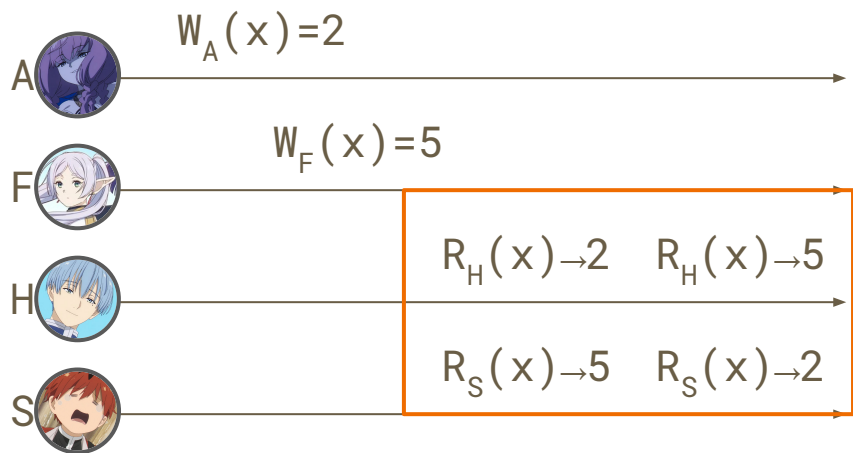
Consistencia Secuencial [Lamport, 1979]

- ◆ El resultado de cualquier ejecución es el mismo que si las operaciones de todos los nodos se ejecutarán **en algún orden secuencial** ... y cada nodo individual respeta dicho orden.
 - ◆ Cualquier intercalación válida de las operaciones entre nodos es aceptable... pero todos son **consistentes con esa** misma intercalación.



Consistencia Secuencial [Lamport, 1979]

- ◆ El resultado de cualquier ejecución es el mismo que si las operaciones de todos los nodos se ejecutarán **en algún orden secuencial** ... y cada nodo individual respeta dicho orden.
 - ◆ Cualquier intercalación válida de las operaciones entre nodos es aceptable... pero todos son **consistentes con esa** misma intercalación.



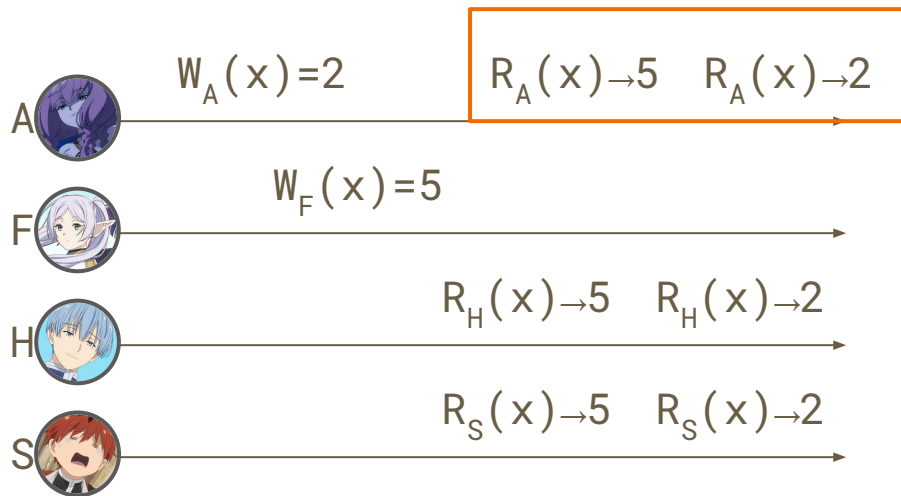
No es consistente secuencialmente.

¿Cuál fue primero, $W_A(x) = 2$ o $W_F(x) = 5$?

No todos los nodos ven el mismo orden.

Consistencia Secuencial [Lamport, 1979]

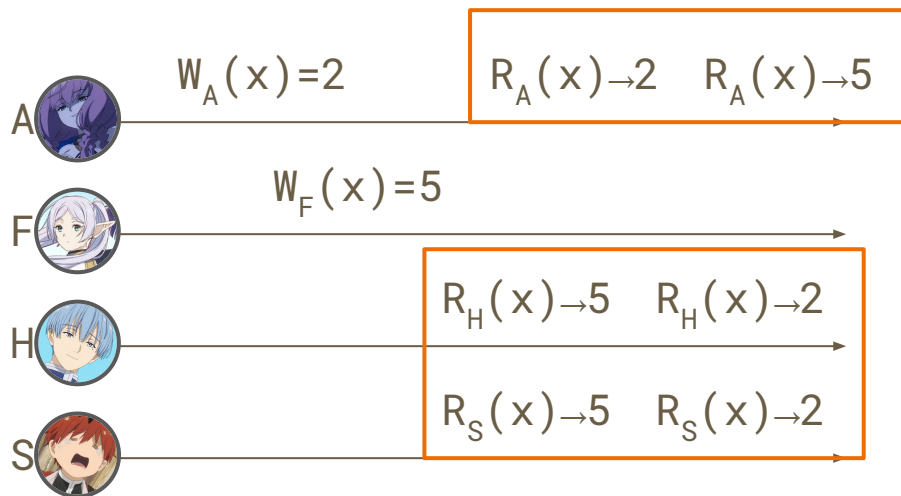
- ◆ El resultado de cualquier ejecución es el mismo que si las operaciones de todos los nodos se ejecutarán en algún orden secuencial ... y cada nodo individual respeta dicho orden.
 - ◆ Cualquier intercalación **válida** de las operaciones entre nodos es aceptable... pero todos son consistentes con esa misma intercalación.



No es válido que el nodo A logró leer 2 después del 5, pero no hay ningún *write* que lo justifica.

Consistencia Secuencial [Lamport, 1979]

- ◆ El resultado de cualquier ejecución es el mismo que si las operaciones de todos los nodos se ejecutarán en **algún orden secuencial ... y cada nodo individual respeta dicho orden.**
 - ◆ Cualquier intercalación **válida** de las operaciones entre nodos es aceptable... pero todos son consistentes con esa misma intercalación.



Nodo A indica que 5 se escribe después de 2, pero nodo H y S dan a entender lo opuesto.

No hay una secuencia de operaciones que todos los nodos respeten

Consistencia Secuencial [Lamport, 1979]

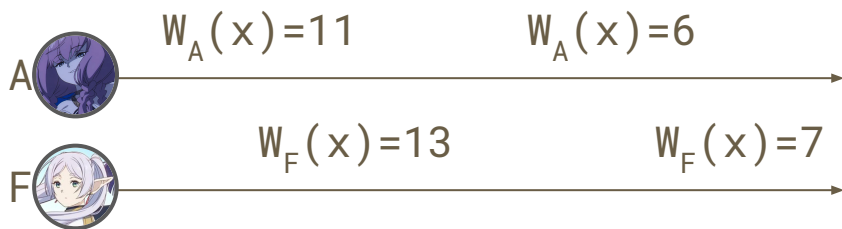
Resumen

El resultado final es como si las operaciones de todos los nodos se hubieran entrelazado de alguna manera secuencial, preservando el orden individual de cada nodo.

Consistencia Secuencial [Lamport, 1979]

Resumen

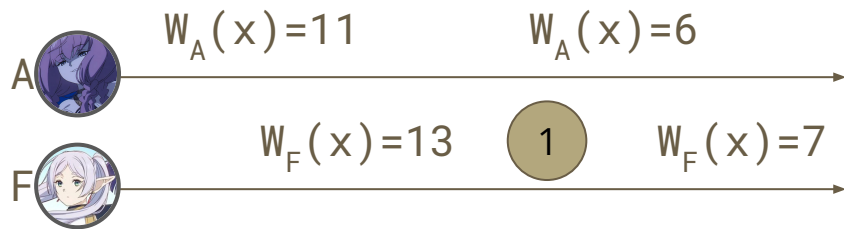
El resultado final es como si las operaciones de todos los nodos se hubieran entrelazado de alguna manera secuencial, preservando el orden individual de cada nodo.



Condiciones mínimas para no romper esta consistencia:

1. $x=11$ viene antes que $x=6$
2. $x=13$ viene antes que $x=7$
3. Todos los *reads* existentes vean la información respetando el mismo orden.

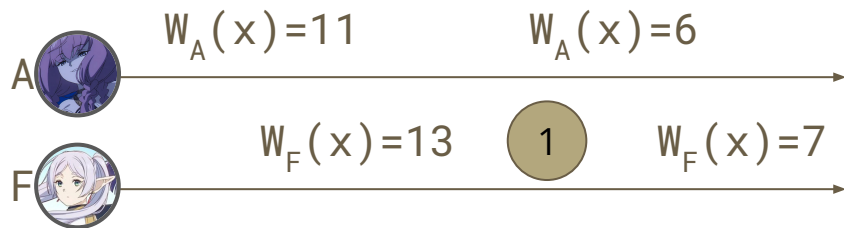
Consistencia Secuencial [Lamport, 1979]



El resultado final es como si las operaciones de todos los nodos se hubieran entrelazado de alguna manera secuencial, preservando el orden individual de cada nodo.

¿Qué valor puede tomar X en (1)?

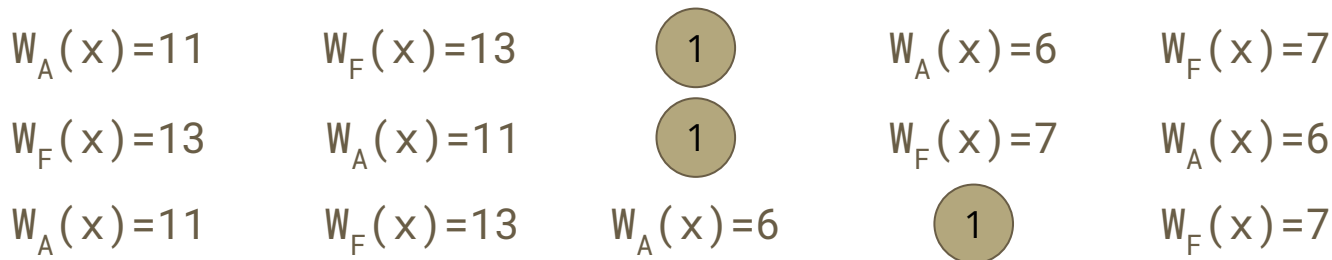
Consistencia Secuencial [Lamport, 1979]



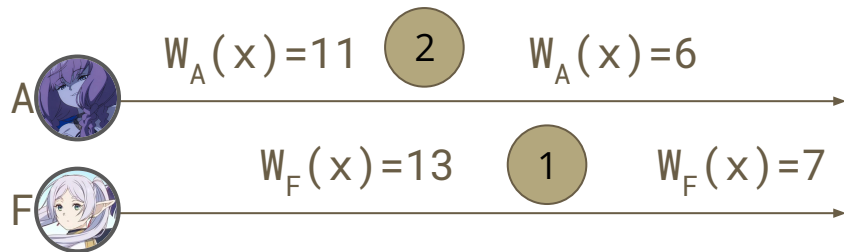
El resultado final es como si las operaciones de todos los nodos se hubieran entrelazado de alguna manera secuencial, preservando el orden individual de cada nodo.

¿Qué valor puede tomar X en (1)?

11, 13 o 6



Consistencia Secuencial [Lamport, 1979]

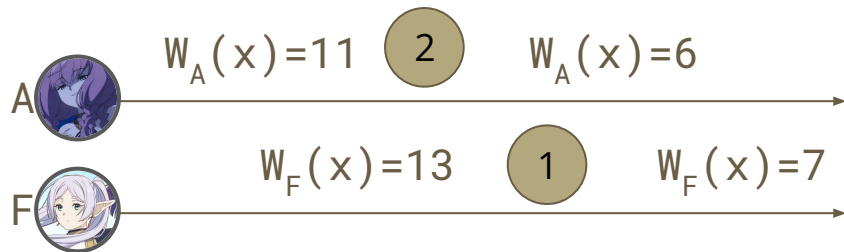


El resultado final es como si las operaciones de todos los nodos se hubieran entrelazado de alguna manera secuencial, preservando el orden individual de cada nodo.

Si (2) es $R_A(x) = 13$

¿Qué valor puede tomar X en (1) ?

Consistencia Secuencial [Lamport, 1979]

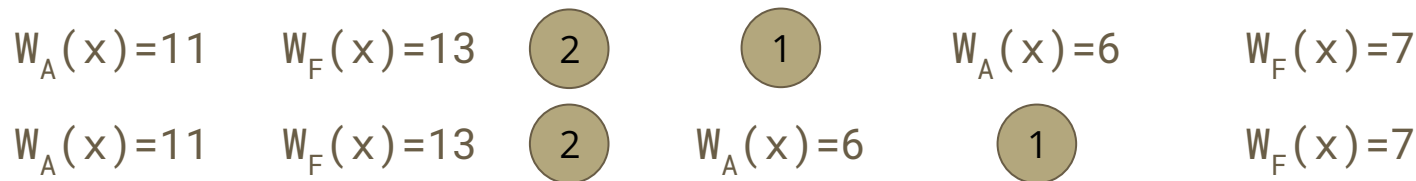


El resultado final es como si las operaciones de todos los nodos se hubieran entrelazado de alguna manera secuencial, preservando el orden individual de cada nodo.

Si (2) es $R_A(x) = 13$

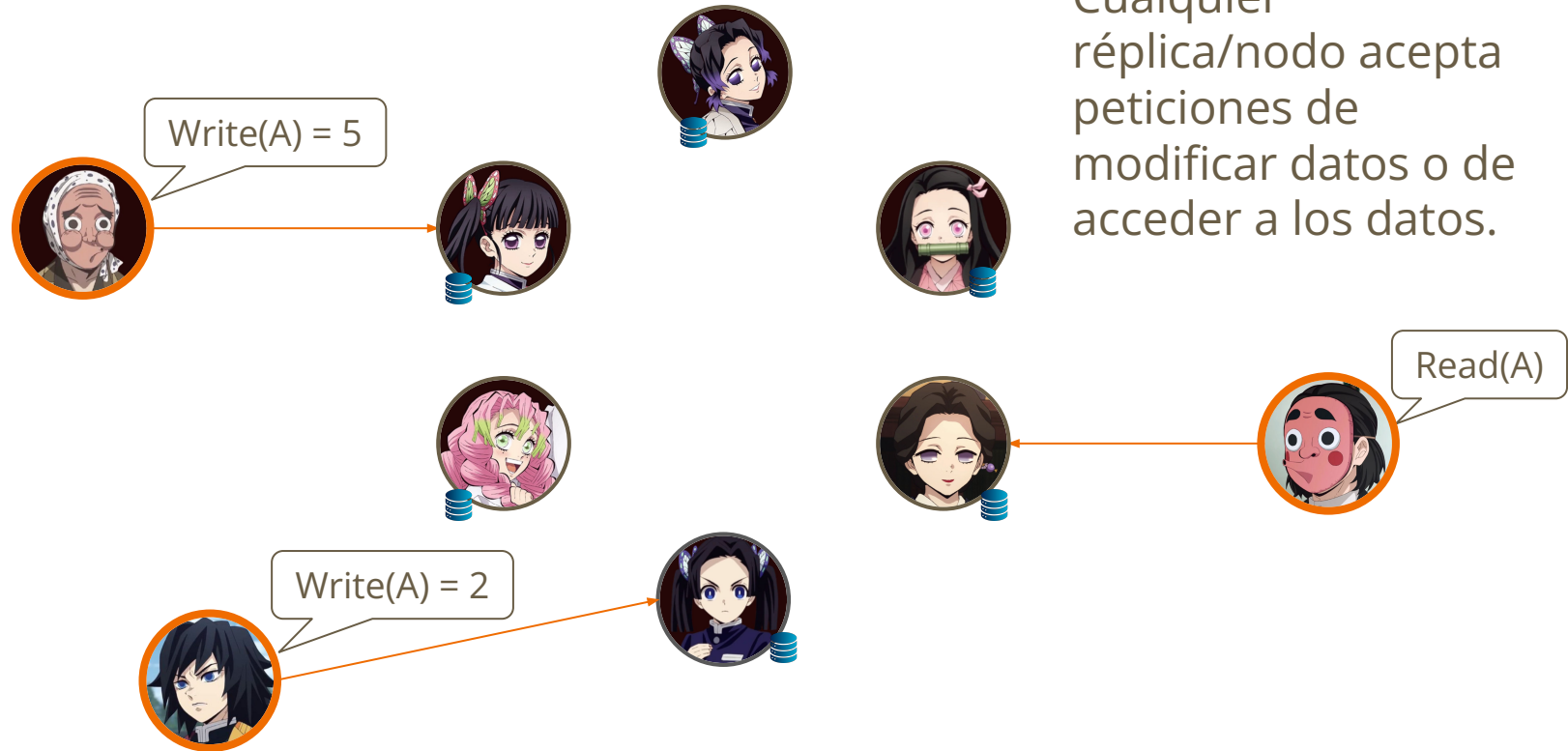
¿Qué valor puede tomar X en (1)?

13 o 6



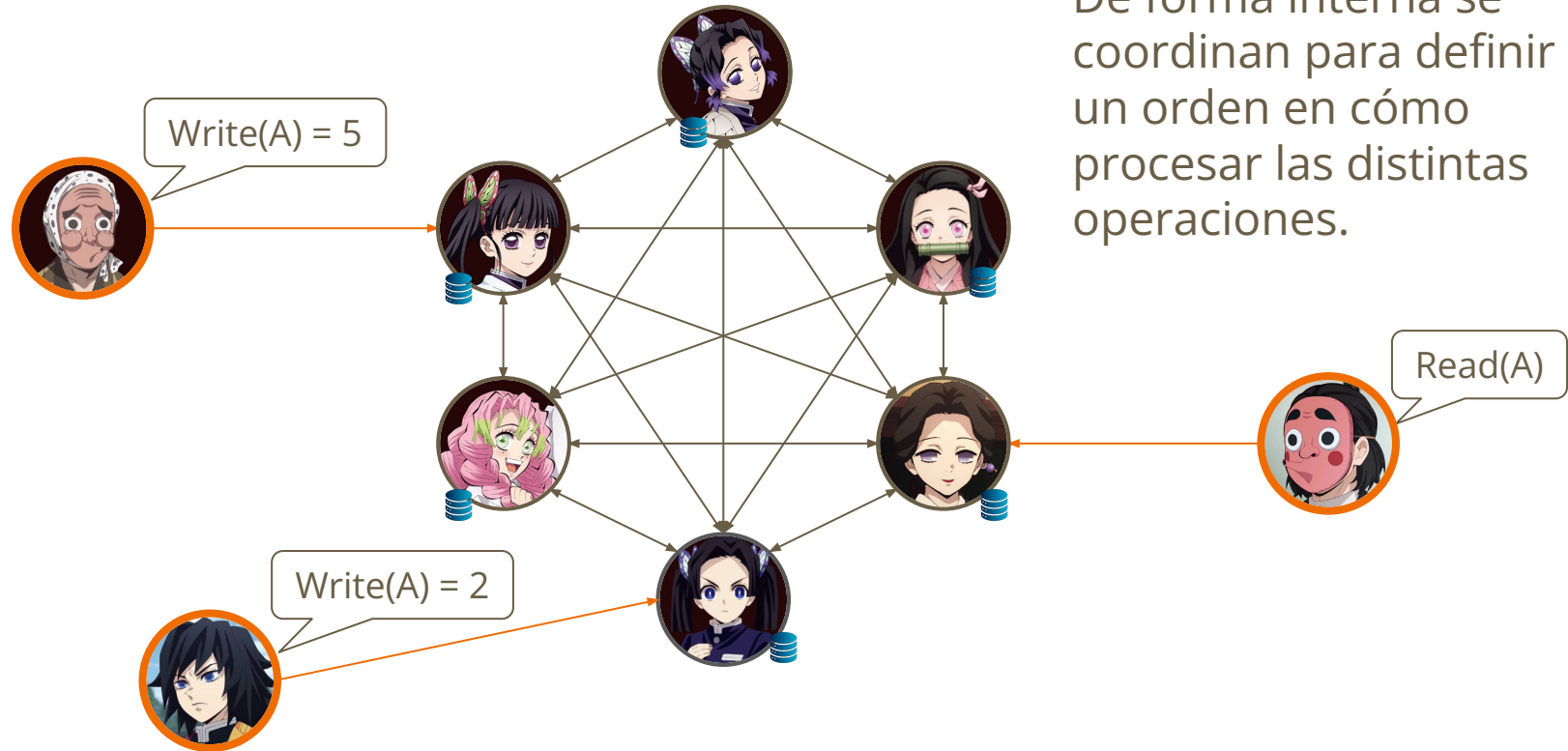
Protocolos de Replicación - *Replicated-Write Protocols*

Protocolos de Replicación - *Replicated-Write Protocols*



Cualquier
réplica/nodo acepta
peticiones de
modificar datos o de
acceder a los datos.

Protocolos de Replicación - *Replicated-Write Protocols*



Protocolos de Replicación - *Replicated-Write Protocols*

- ◆ También conocido como Replicación Activa.
- ◆ Todos los nodos están activos y procesan las solicitudes de los clientes de forma concurrente.
- ◆ Para garantizar la consistencia entre réplicas, se utiliza un mecanismo de *Totally Ordered Multicast (TOM)*.
 - ◆ Mediante algoritmo de consenso o con un coordinador, se asigna un número de secuencia global a la operación.
 - ◆ Este número de secuencia debe respetar el orden temporal dentro de un mismo nodo. Si *write(x)* ocurre antes que *write(y)* en un nodo, entonces el número de *write(x)* será menor a *write(y)*.
 - ◆ Cada nodo ejecuta las operaciones respetando el número de secuencia.

Protocolos de Replicación - *Replicated-Write Protocols*

Consistencia

- ◆ Cada nodo verá las operaciones que ocurrieron en su réplica en el orden correcto.
- ◆ Cada operación concurrente se asignará un orden global y serán ejecutadas por cada nodo en ese orden.
- ◆ Aunque temporalmente una operación $write_{nodo\ 1}$ ocurre antes que $write_{nodo\ 2}$, si son determinadas como concurrentes, el *Totally Ordered Multicast* puede indicar que $write_{nodo\ 2}$ se procese antes que $write_{nodo\ 1}$, y todos los nodos procesarán las operaciones en dicho orden.
- ◆ **Se garantiza la consistencia secuencial.**

Protocolos de Replicación - *Replicated-Write Protocols*

Actualidad

- ◆ Se tiende a recurrir más a elección de líder y este es el único encargado de autorizar y ordenar las operaciones.
 - ◆ No es que el nodo líder las ejecute, solo les asigna un orden.
- ◆ Sin un líder, igual se puede recurrir a protocolos que requieran quórum (como paxos) para llegar a un consenso en el orden.
- ◆ *Practical Byzantine Fault Tolerance* (pBFT) se inspira en *Replicated-Write Protocols* para aceptar o no el cambio.

Protocolos de Replicación - *Replicated-Write Protocols*

Ventajas

- ◆ Hay una alta tolerancia a fallos ante caída de algunos nodos.

Desventajas

- ◆ Costo de coordinación alto.
- ◆ Escalabilidad limitada.
- ◆ Latencia de escritura alta.
- ◆ Alta tasa de mensajes.

Problemas con la consistencia secuencial

Muchas opciones válidas que son secuencialmente consistente.



Existen 6 formas de ordenar $W_A(x)=1$ $W_A(y)=1$ $W_F(y)=2$ $W_F(x)=2$ que serán **secuencialmente consistentes**.

Problemas con la consistencia secuencial

Muchas opciones válidas que son secuencialmente consistente.



Existe
secue

$W_A(x)=1$	$W_A(x)=1$	$W_F(y)=2$	$W_A(x)=1$	$W_F(y)=2$	$W_F(y)=2$
$W_F(y)=2$	$W_A(y)=1$	$W_F(x)=2$	$W_F(y)=2$	$W_A(x)=1$	$W_A(x)=1$
$W_F(x)=2$	$W_F(y)=2$	$W_A(x)=1$	$W_A(y)=1$	$W_F(x)=2$	$W_A(y)=1$
$W_A(y)=1$	$W_F(x)=2$	$W_A(y)=1$	$W_F(x)=2$	$W_A(y)=1$	$W_F(x)=2$

Modelo de Consistencia Fuerte

¿En qué consiste?

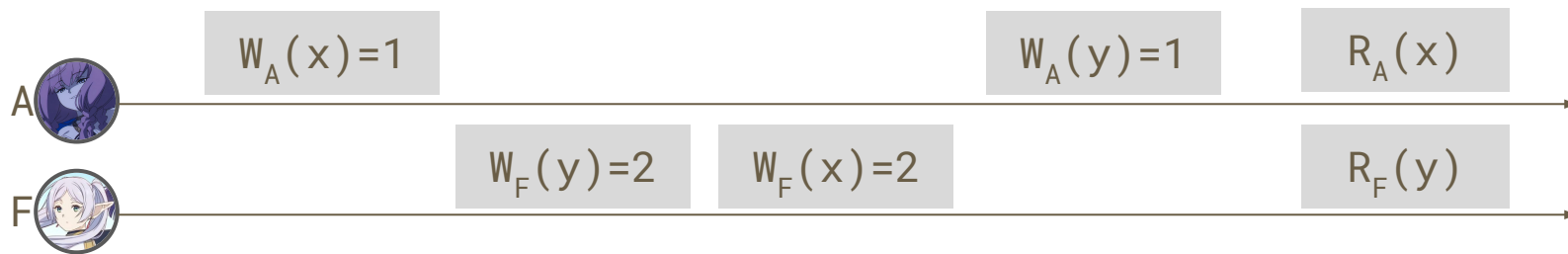
Protocolo de replicación Pasiva
(*Primary-Based Protocols*)

Consistencia Fuerte [Herlihy & Wing, 1986]

- ◆ Consistencia más estricta que la secuencia. Llamada también **Linealizabilidad**.
- ◆ Cada operación debe aparecer como que tiene efecto instantáneo en algún momento entre su inicio y su finalización.
 - ◆ La consistencia fuerte está **totalmente ligado al tiempo real de las operaciones**.
- ◆ Al momento que se completa un *write*, esta operación debe ser propagada inmediato a las demás réplicas que pueden hacer *read*.

Consistencia Fuerte [Herlihy & Wing, 1986]

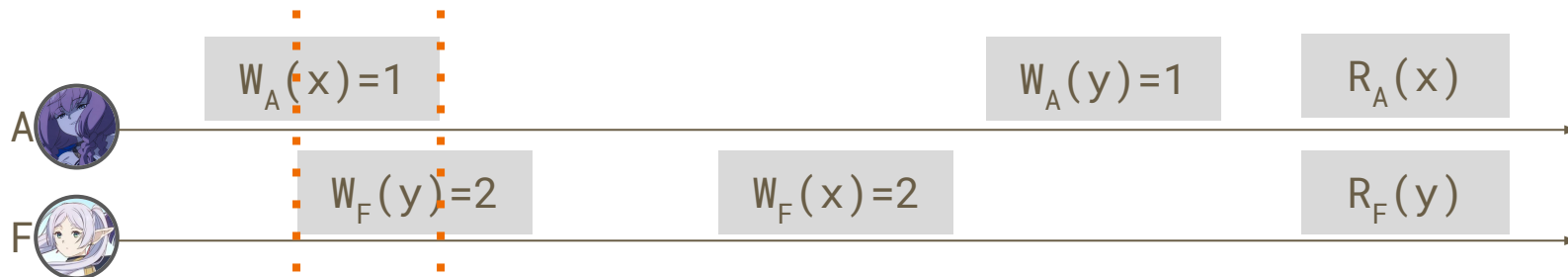
- ◆ Consistencia más estricta que la secuencia. Llamada también **Linealizabilidad**.
- ◆ Cada operación debe aparecer como que tiene efecto instantáneo en algún momento entre su inicio y su finalización.
 - ◆ La consistencia fuerte está **totalmente ligado al tiempo real de las operaciones**.
- ◆ Al momento que se completa un *write*, esta operación debe ser propagada inmediato a las demás réplicas que pueden hacer *read*.



Solo 1 secuencia válida de *writes*

Consistencia Fuerte [Herlihy & Wing, 1986]

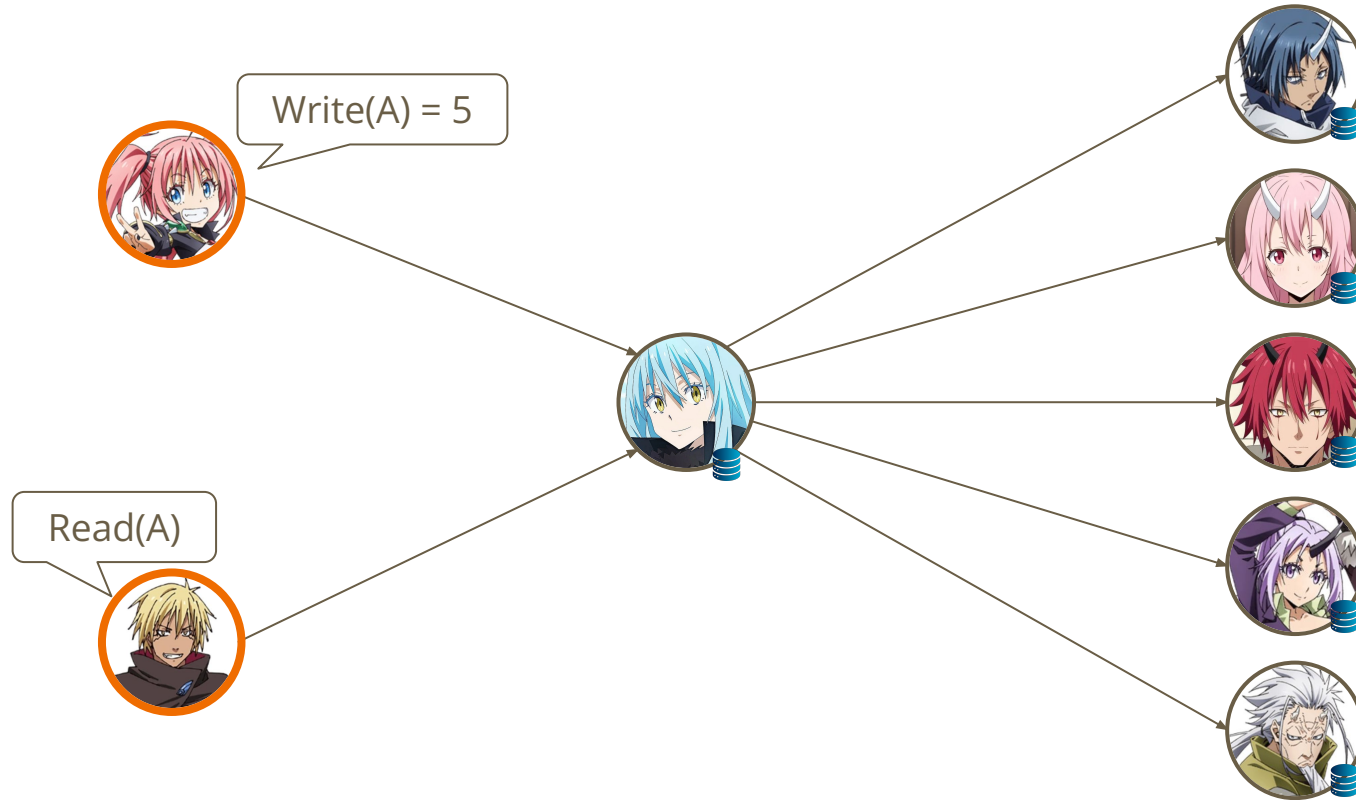
- ◆ Consistencia más estricta que la secuencia. Llamada también Linealizabilidad.
- ◆ Cada operación debe aparecer como que tiene efecto instantáneo en **algún momento entre su inicio y su finalización**.
 - ◆ La consistencia fuerte está **totalmente ligado al tiempo real de las operaciones**.
- ◆ Al momento que se completa un *write*, esta operación debe ser propagada inmediato a las demás réplicas que pueden hacer *read*.



Solo 2 secuencia válida de *writes*

Partir con $W_a(x)=1$ o partir con $W_f(y)=2$

Protocolos de Replicación - *Primary-Based Protocols*



Protocolos de Replicación - *Primary-Based Protocols*

- ◆ También conocido como Replicación Pasiva
- ◆ Este protocolo se basa en un único gestor de réplicas primario que recibe todas las solicitudes de los clientes.

Protocolos de Replicación - *Primary-Based Protocols*

- ◆ También conocido como Replicación Pasiva
- ◆ Este protocolo se basa en un único gestor de réplicas primario que recibe todas las solicitudes de los clientes.
- ◆ El primario ejecuta las operaciones y luego propaga **el nuevo estado** a los gestores de réplicas secundarios.
 - No necesariamente propaga cada *write*, sino que propaga los mínimos cambios necesarios.
 - Si hace $\text{Write}(X) = 2$, $\text{Write}(X) = 3$ y $\text{Write}(X) = 4$ en un tiempo muy acotado de tiempo, solo necesita propagar que actualicen x a 4, sin pasar por $x=2$ o $x=3$.

Protocolos de Replicación - *Primary-Based Protocols*

Consistencia

- ◆ Si la lectura y escritura se realiza exclusivamente con el nodo primario, se garantiza **consistencia fuerte**.

Protocolos de Replicación - *Primary-Based Protocols*

Consistencia

- ◆ Si la lectura y escritura se realiza exclusivamente con el nodo primario, se garantiza **consistencia fuerte**.
- ◆ ¿Y si las lecturas se pueden hacer a alguna réplica?



Protocolos de Replicación - *Primary-Based Protocols*

Consistencia

- ◆ Si la lectura y escritura se realiza exclusivamente con el nodo primario, se garantiza **consistencia fuerte**.
- ◆ ¿Y si las lecturas se pueden hacer a alguna réplica?
- ◆ Depende:
 - Si la operación de escritura solo se declara completada cuando sus cambios fueron propagados, se mantiene consistencia fuerte.
 - Si la operación de escritura se declara completada cuando solo el nodo primario la procesa, solo se garantiza consistencia secuencial.

Protocolos de Replicación - *Primary-Based Protocols*

Ventajas

- ◆ Eficiencia en escrituras consecutivas. Se empaquetan varias operaciones como una gran operación y transmitiendo a las réplicas sólo el estado final.
- ◆ Relajando la consistencia, se puede separar el proceso de escritura y lectura para reducir la carga del nodo primario.
- ◆ Excelente manejo de operaciones no deterministas
 - Operaciones cuyo resultado puede cambiar entre una ejecución y otra porque dependen del momento de ejecución como el tiempo o de un random
 - Solo se propaga el cambio final sin necesidad de ejecutar más de una vez la operación.

Protocolos de Replicación - *Primary-Based Protocols*

Desventajas

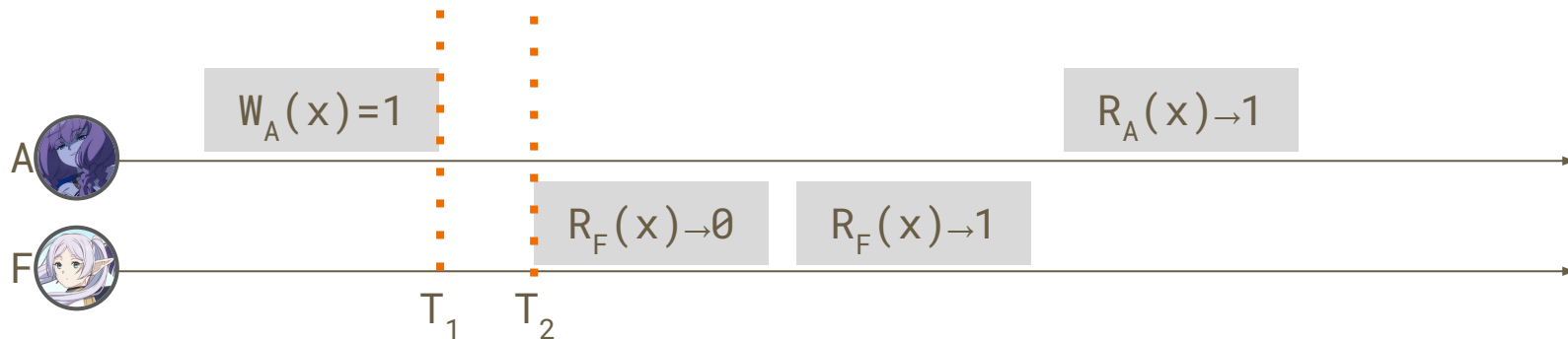
- ◆ **Punto único de fallo** (*Single Point of Failure*): Se depende que la réplica principal esté activa para que todo funcione.
- ◆ **Cuello de botella en escrituras**: Todas las modificaciones deben pasar obligatoriamente por el nodo primario. Limita la capacidad de escalar las peticiones de escritura de forma concurrente.
- ◆ **Latencia elevada**: Las escrituras bloqueantes (aquellas que esperan la confirmación de las copias de respaldo) aumentan el tiempo de respuesta general para los clientes.

Protocolos de Replicación

¿Cómo se ven reflejados en la actualidad?

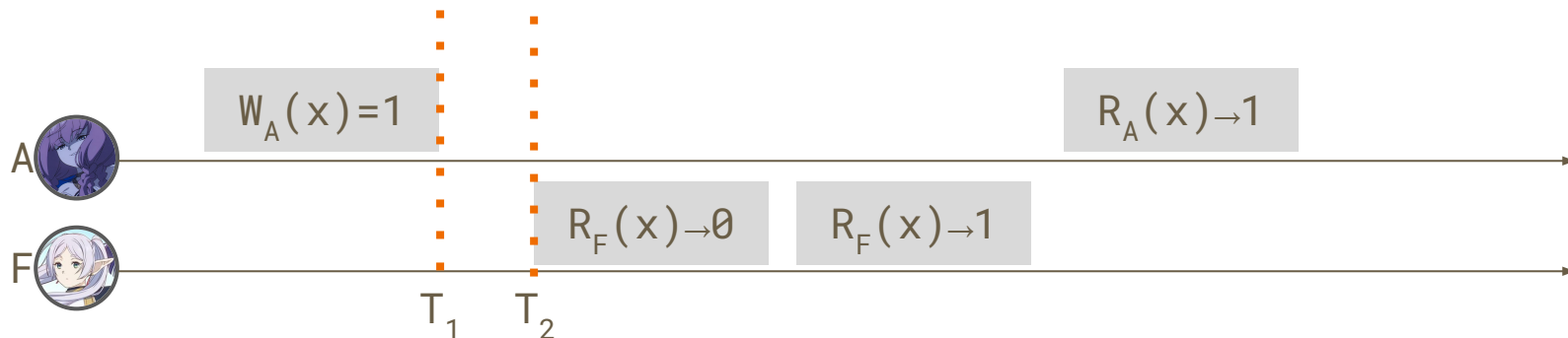
- ◆ [Azure SQL Database](#) utiliza *Primary-Based Protocols*, de forma asíncrona, para replicar la información en hasta cuatro réplicas geográficas.
- ◆ [MySQL Server](#) utiliza *Primary-Based Protocols* para replicar los datos. Puede ser asíncrono (responde antes de replicar) o síncrono (responder después de replicar).
- ◆ [Google file system \(GFS\)](#) utiliza *Primary-Based Protocols* para garantizar la consistencia, especialmente para las mutaciones de datos. No obstante, exige que todas las réplicas hayan completado con éxito la mutación antes de indicar que la operación fue exitosa.

Poniendo a prueba lo que hemos aprendido 🧐🧐



◆ Asumiendo $T_1 < T_2$ ¿Se cumple algún modelo de consistencia?

Poniendo a prueba lo que hemos aprendido 🧐🧐



- ◆ Asumiendo $T_1 < T_2$ ¿Se cumple algún modelo de consistencia?
- ◆ Es consistente secuencialmente, pero no fuerte.
- ◆ Puede ser que nodo F leyó la variable X antes que le llegara la propagación del cambio de X por el nodo A.
 - ◆ Existe una secuencia **válida secuencial de operaciones** ($R_F \rightarrow W_A \rightarrow R_F \rightarrow R_A$)
 - ◆ No respeta el requisito de **tiempo real** de la consistencia fuerte.

Próximos eventos

Próxima clase

◆ Replicación de Datos II

- ◆ ¿Podemos relajar la consistencia de los datos para optar por sistemas más eficientes?

Evaluación

- ◆ **Control 11 - Obligatorio** - 2 preguntas sobre la clase de hoy. Se puede responder hasta este viernes a las 20:00.

IIC2523

Sistemas Distribuidos

— Hernán F. Valdivieso López —
(2026 - 2 / Clase 11)

Créditos (animes utilizados)

Sōsō no Frieren



Tensei shitara Slime Datta Ken



Kimetsu no Yaiba

